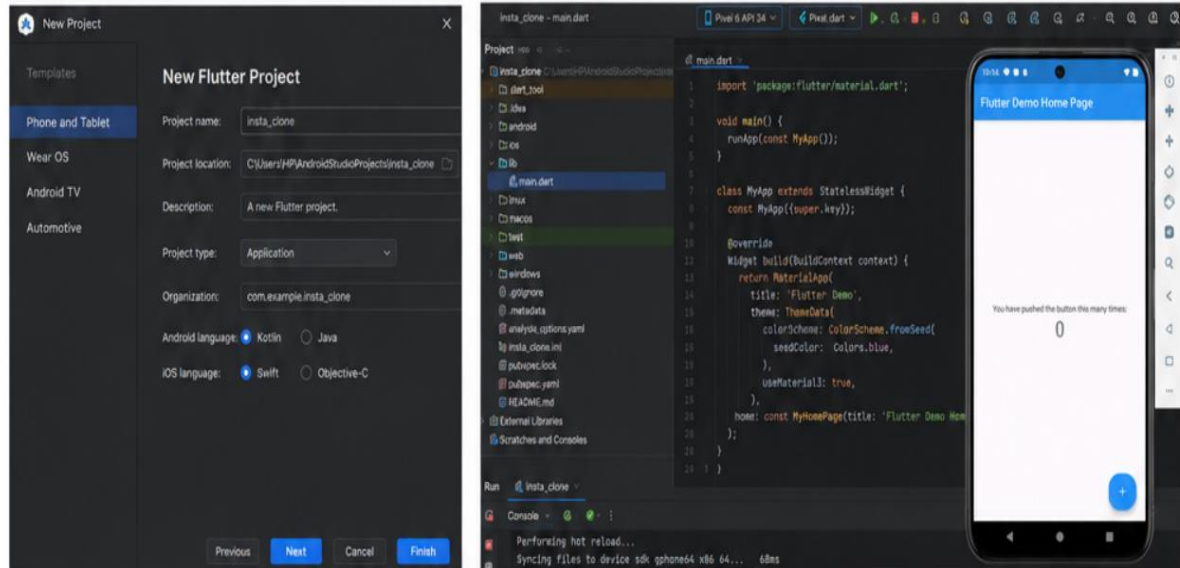
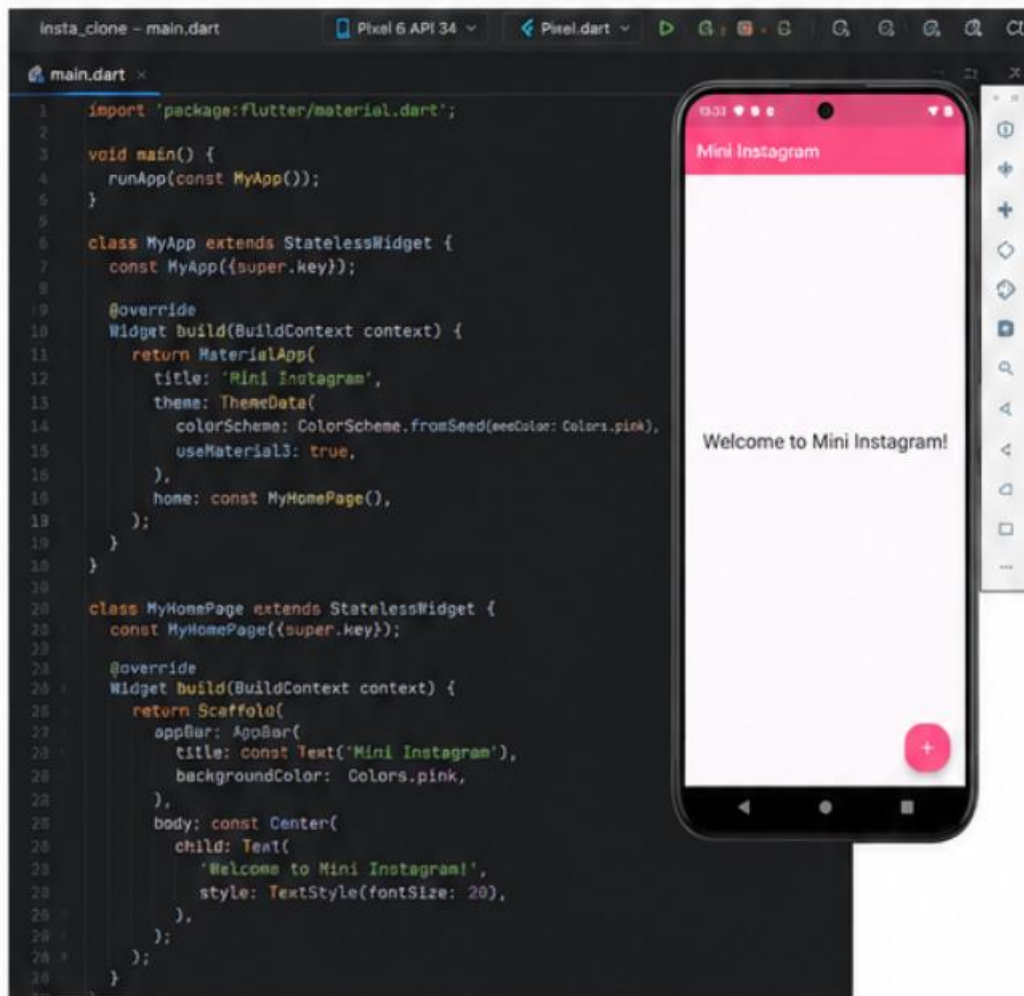


1.

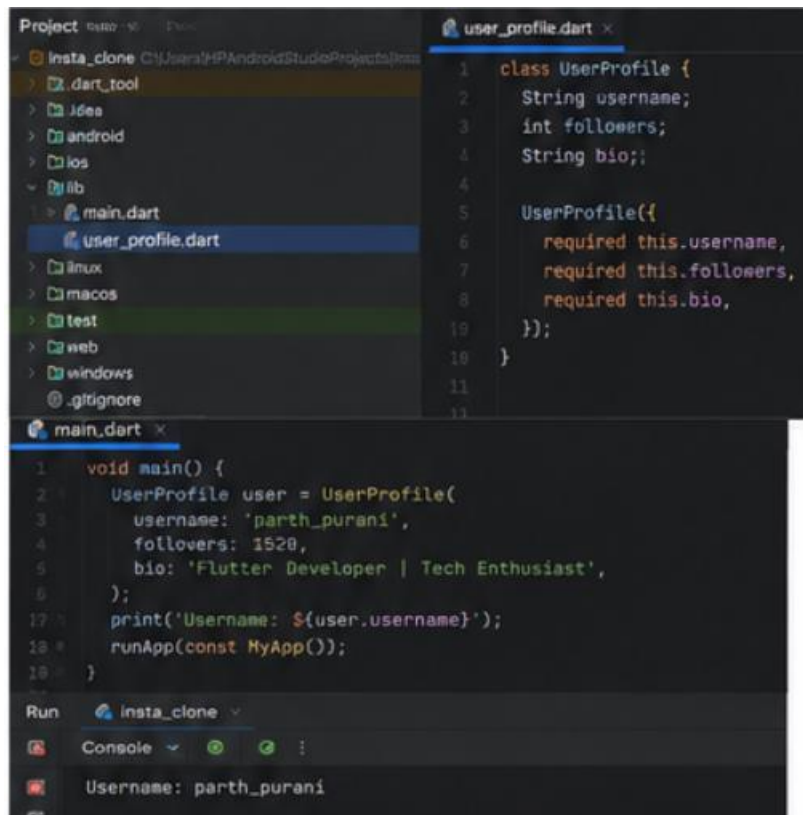
Install Flutter and Dart SDK on your system, create a new Flutter project named `insta_clone`, and run it to display the default counter app in your emulator or physical device.



2. Inside your insta_clone project, edit main.dart to change the app title to 'Mini Instagram' and update the home screen text to 'Welcome to Mini Instagram!'.



3. Add a new Dart file called `user_profile.dart` in the `lib` folder and define a `UserProfile` class with properties: `username` (`String`), `followers` (`int`), and `bio` (`String`). Create an instance of `UserProfile` in `main.dart` and print the `username` in the debug console.



The screenshot shows an IDE with two files open. The top file, `user_profile.dart`, contains the definition of the `UserProfile` class. The bottom file, `main.dart`, contains the `main` function where an instance of `UserProfile` is created and its `username` property is printed. The bottom panel shows the console output: `Username: parth_purani`.

```
1 class UserProfile {
2   String username;
3   int followers;
4   String bio;
5
6   UserProfile({
7     required this.username,
8     required this.followers,
9     required this.bio,
10  });
11 }
12
```

```
1 void main() {
2   UserProfile user = UserProfile(
3     username: 'parth_purani',
4     followers: 1520,
5     bio: 'Flutter Developer | Tech Enthusiast',
6   );
7   print('Username: ${user.username}');
8   runApp(const MyApp());
9 }
```

Run: insta_clone

Console: Username: parth_purani

4. Write a Dart function `formatFollowers(int count)` that returns '1.5K' for 1500, '2M' for 2000000, or the actual number for less than 1000, similar to how Instagram displays follower counts.

Hint: Use conditional statements and string interpolation for formatting.

```
String formatFollowers(int count) {
  if (count >= 1000000) {
    double millions = count / 1000000;
    return '${(millions.toStringAsFixed(millions % 1 == 0 ? 0 : 1))}M';
  } else if (count >= 1000) {
    double thousands = count / 1000;
    return '${(thousands.toStringAsFixed(thousands % 1 == 0 ? 0 : 1))}K';
  } else {
    return '$count';
  }
}
```

Run Insta_clone ▾

Console ✓ ✓ ⋮

500	->	500
1500	->	1.5K
5230	->	5.2K
2000000	->	2M

5. List 3 ways to validate if your Flutter environment is set up correctly and describe what output or result you expect from each validation step.

Validation Step		Command / Action	Expected Output / Result
1.	Check Flutter installation	<code>flutter --version</code>	- Flutter and Dart version details - Example: Flutter 3.16.5 • channel stable • https://github.com/flutter/flutter.git Framework • revision abc123 (5 weeks ago) Engine • revision def456 Tools • Dart 3.2.3 • DevTools 2.28.4
2.	Check Flutter environment	<code>flutter doctor</code>	- All checks marked with ✓ - No errors in red - Example: [✓] Flutter (Channel stable, 3.16.5, on Windows 11) [✓] Android toolchain - develop for Android devices (Android SDK version 34.0.0) [✓] Chrome - develop for the web [✓] Android Studio (version 2023.2) [✓] Connected device (2 available) - No issues found!
3.	Run Flutter sample app	<code>flutter run</code>	- App builds and launches successfully - Default counter app appears in emulator or connected device - Console shows: "Performing hot reload..." "Application running."